

Weekly Notebook

Week 0

01/07/25

The overall goal of last semester was to propose a platform or interface to help students form study groups and study better. How Ai can be used to gather resources and facilitate interactions and successful groups.

Grades are based on participation in class, notebook, and evaluations of each other. How does your journal tell the story of what you contributed to your team.

This class will focus more on development and secondary and primary research.

2 evaluations, Midterm and Final (if any one of these are not received, you will be marked down a letter grade). This is VIP policy.

Schedule

02.02 - midterm notebooks due (email)

03.18 - no class

04.15 - demo and notebooks due

04.22 - last day of class

Notebooks: dates, checkboxes and dates for completion.

Best practice: bound notebook, name/team/semester/contact info on cover, number your pages, table of contents of index, to do lists, meeting notes.

Next week get your team members contact information.

Crowdsourcing - an approach to performing a particular task that divides work amongst a set of crowd participants. tasks can fall under multiple categories such as asking for information/opinions/data/predictions/hypotheses

Steps to implementation:

- ☐ creating an interface
- ☐ storing users and their data
- ☐ hosting the app
- ☐ creating a survey (containing the best and most efficient characteristics for successful matching)
- ☐ an algorithm (using AI API calls and/or hard coded social psychology best practices) for matching
- ☐ social interface (by course and by group) for resources
- ☐ direct message/study group chats interface, and prompts/activities to facilitate interactions
- ☐ tagging system for specific classes and

First day thoughts:

The interface could be similar to reddit and combine an ed discussion/piazza feature by subject.

Week 1

01/14/25

Choosing teams and roles!

I would be interested in making the interface. The team that best suits this best may be Managing Resources and Extracting Value.

I feel like its hard to conceptualize the backend without the frontend implementation, but we can work on a matching algorithm that calls an AI API and formats a response that can be parsed by a program to sort people into relevant groups.

This VIP is about designing a prototype with your group.

<https://www.youtube.com/watch?v=rCm5RVYKWVg>

<https://www.youtube.com/watch?v=FCtJgz1Gg8k>

“Pay no attention to the man behind the curtains” - last semester
this semester the man is out and we are paying attention to him 😭

Managing Resources

What are our resources?

- past/current class notes
- past exams
- course materials
- questions and answers

How should they be grouped? 1st priority - grouping by course/subject

- approved by course staff
- sorted by upvotes/downvotes
- If the poster (forgive my reddit terminology) has high karma and is tagged as a good resource by other users

How does this facilitate the other functionalities of the application?

Other app functionalities:

- Sorting users into study groups
- Facilitating interactions (ensuring that people are active and have a good conversion rate to IRL activities)

Intelligent Features:

- AI sentiment analysis and tagging submitted resources so they're easier to search
- AI chatbot that can answer questions and link the asker to the submitted resources
- AI identifies the topic of the thread and sends a notification to top contributors in that subject to add to it.

As a VIP as a whole, what platform and languages do we want to use?

- i would say Django because that is what our group is accustomed to (CS2340)
- javascript/python3/html/css

Team Milestones

B	C	D	E
Date	Class Milestones	Team Milestones - Managing Resources	Team Milestones - Managing Resources
1/7/2025	Introductions Discussion of semester goals		
1/14/2025	Prepare VIP notebook (Bring to all future classes). Break into teams to prototype existing work	Get on same page about roles & goals	
1/21/2025	Guest speaker		
1/28/2025		Basic interface that can create and group posts by subject	
2/4/2025			
2/11/2025	* In class demo of prototypes * Collect notebooks	AI sentiment analysis program for tagging submitted resources	
2/18/2025	* Turn in test plan * Collect notebooks	Create test data (i.e. example posts and fake users)	
2/25/2025	Conduct test		
3/4/2025	* Share test findings and analysis * Identify next steps		
3/11/2025			
3/18/2025	Spring Break (March 17-March 21)		
3/25/2025			
4/1/2025	Prepare for application demo		
4/8/2025	Prepare for application demo		
4/15/2025	* Demo application * Collect notebooks		
4/22/2025	* Lessons learned * Return notebooks		

01/28/2025 Team Questions

Prototype goal & key features

1. How are you defining top contributions, users who need encouragement sharing, and posts that merit badges?

Top contributors are the ones who have received the most upvotes, and have been tagged as a good resource by course organizers

2. What data will the prototype use to determine the above?

user contributed data

3. Will AI be used to determine the above? If so, how?

AI will be used to analyze the submitted documents and make it user friendly to find the best resources.

4. How will LLM be use to organize content by topic?

the users themselves can tag the topics when they post it an it can also work that the AI is provided the syllabus and then is able to tag the documents itself

5. Are you still planning on conducting sentiment analysis using LLM?

yes, the ai will review the documents and in future implementations be able to tag posts if trained on user interactions with the posts

User flows & wireframes

6. What are the user scenario(s)/user flow(s) that you want to demonstrate through the prototype?

we would like to demonstrate the tagging and post making process of which the user info and post content is run through an llm api to summarize the quality of the information

7. Are you aiming for interactive prototype or just visual walk through?

we are aiming for an interactive UI but feature walkthrough

Test data, data flow, dev environment

8. What data do you plan to use to test the prototype? (i.e., real or synthetic)

- Input data vs. Output data
 - real ed discussion posts
 - open source classes, where the ai is given the link and is able to read it itself and provide hte sentiment analysis
- User prompts vs. System prompts
 - user prompts for making the posts and the system automatically does the sentiment analysis for the resource

9. How are you going to obtain your dataset?

internet resources

10. What is your target sample size and attributes of the dataset?

3-4 posts

Prototype testing

1. What do you want to test? Define clear and measurable objectives to ensure your test plan will yield actionable

we will test whether the ai tags the resource appropriately. and sorts it by relevance

test results. For example:

- I want to test how well the algorithm/AI determines top contributors based on x, y, z measures... (look through what quantitative things you can measure, or compare it to how use humans would rank it)
- I want to test the quality of the prompts used to organize posts based on x, y, z measures..

I will be making the UI 🐱

professor suggestion: use subreddits as classes

think about prompt performance

Creating the “StudyBuzz” web app (my contribution)

used flask framework

language: Python

Features:

- User Management (create a model with username and password and an array to store posts by id)
- Display posts
- Organize posts by course
 - can still add a drop down or the poster can create them and it adds to a database
- Tag posts (on user and viewer ends)

Integration:

- the tags will be in a database and associated with posts, and will be able to be fed into our sentiment analysis.
- we can use a library/api to analyze documents (typically library dependencies are not best practice but do i look like bill gates yall....) this will only be for pdf/txt files because images can use an llm. This would lowkey only be if there was a reasonable funding source for the application.
- integrate with messaging feature of identifying connections group

StudyBuzz Create Post Logout

Create a New Post

Title

Course

Content

Tags
 Add Tag

Add Media
Images (PNG, JPG, GIF)
Choose Files No file chosen

Documents (PDF, DOC, DOCX)
Choose Files No file chosen

Add Photo
Choose File No file chosen

The post content will be processed as text and the photo (if uploaded will be the cover photo for the post) and run through the sentiment analysis

this imo should be judged on a couple things

length/complexity (judged wholly by the llm)

if there are additional uploaded materials (even if we cannot analyze them)

The llm could

2/4/25

GTRI intern - Daniel Osagie (senior at KSU) REI Co-op

Special Guest Speaker

development | prototyping | fast iterations

go as simple as possible for required data - stored in supabase (open source google firebase)

create with play .com

design inspiration practicedesign.io

mobbin

take inspiration from established companies because the research and resources have been put into it.

shadcn documentation

LLM providers - Groq/Ollama/huggingface

o/h you can keep your data and totally customize it.

TRAE is like JetBrains but free (yas bytedance 🤗)

V0 development - Vercel and Render

it can render your Figma designs in code.

gadget.dev build platform (fav)

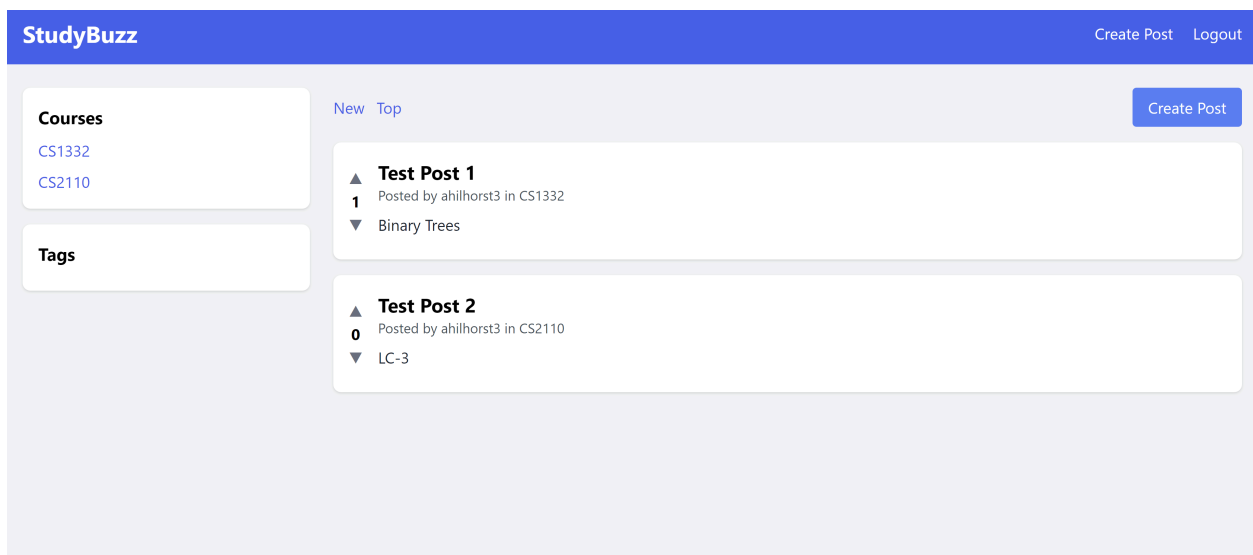
Jitter - animation

Microtask data labelling - Label Studio

Play.ai

Advertising grassroots on social media where your users are

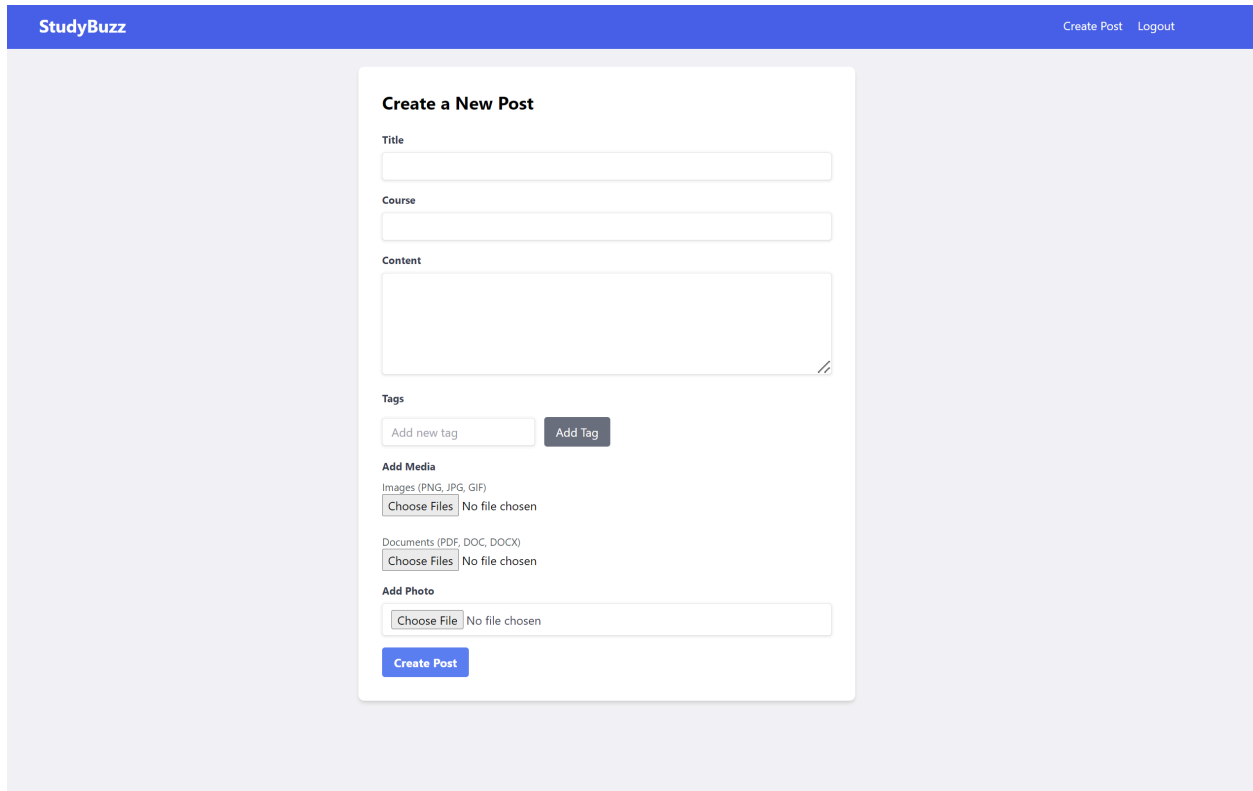
App functionality



Sorts posts by class, and subsequently by tag.

You can order by New and by Top in general or by course

You can give a post upvotes



The screenshot shows the 'Create a New Post' form on the StudyBuzz website. The form is centered on a light gray background. At the top, there is a blue header bar with the 'StudyBuzz' logo on the left and 'Create Post' and 'Logout' links on the right. The form itself is a white card with a title 'Create a New Post'. It contains several input fields: 'Title', 'Course', and 'Content' (a larger text area). Below these are 'Tags' with an 'Add new tag' input and an 'Add Tag' button. There are three sections for media: 'Add Media' for images (PNG, JPG, GIF), 'Documents' (PDF, DOC, DOCX), and 'Add Photo'. Each has a 'Choose Files' button and a 'No file chosen' status. At the bottom of the form is a blue 'Create Post' button.

When creating a post, the fields are for

- title
- course (if course doesn't exist it will create a new column in the db for that course)
- Content is what will be run in the quality analysis as the topic of the post
- tags will be run in the sorting algorithm
- media will be run in the quality analysis
- overall post interaction, poster karma, and document sentiment analysis will determine the ranking of the doc.
- photos will be made into the cover image for the post

StudyBuzz

Login Register

Login

Email

Password

Sign In Register

user management

- posts are stored under a user account
- display username can be changed by course (or the sentiment analysis can give you titles in different courses) (havent done this yet btw)

Continued features:

user tagging for sentiment analysis

allowing users to search or ask questions

Demo suggestions:

training an llm on the study materials so users can ask questions directly to the llm and it will pull up posts with those study materials and answer the questions

how would we do this?

locally run a small model, and whenever a document is submitted, tokenize it, and submit for the llm to learn. this can be done with dual processing and in chunks.

i have never done anything like that before tbh. another option is just calling an llm api prompt, using a library to tokenize/convert the document to text, and then imputting it into a prompt too

the api. I am not sure if this will work for memory recall if you dont use your own model in the database. OpenAI is expensive per prompt, especially for image generation.

i think it would be best for implementing the feature for users to get their questions answered, is just to plug their question into an llm api request, and display the response on the webpages, and simultaneously run a keyword search for posts/documents in our webpage database. this is not api based and will all be done in our own code, keeping dependencies lower. (and i better know how to do this 🐱)

2/18/25 Notes

grok and agenta.ai

prompts and showing the results of the prompts - identify helpful attributes for users to know.

defining quality/accuracy - how consistent to canonical sources (comparing to course notes).

we can use summarization, looking for differences between content.

2/25/25

testing suggestions

evaluate the llms ability to provide a ranking system

how do the prompts perform. limit it to a topic we ourselves know well and can assess.

try to analyze shakespeare (public domain)

problem encountered:

there is no “ground truth” in documents for the quality analysis, i think we should lean more on sentiment analysis for assessing quality.

llms are computationally expensive at application level.

since llms are non deterministic and have varying response temperature.

thoughts:

using sentiment analysis to train an model on what is a good resource, or creating a smaller in-house model

GTRI R&D partnered with fort stewart (in savannah)

helping military spouses - you have to move around a lot (mandatory change of base). It is difficult for military spouses to grow their careers, becoming the primary caregiver etc.

Concept testing - prototype

stakeholder engagement

career development model

thrive toolkit - star bullets

removing uncertainty from your testing

met with senator ossofs office

Testing - 3/4/25

Different prompts and their outcomes

Different Prompt Formats and their Performance

Code Flow Overview

1. GET /submit

Display a form to submit a post (title, content, tags).

2. POST /analyze

Handle form submission, send content to LLM API, receive analysis back.

3. GET /result/<post_id>

Render the post, tags, and quality analysis in a styled results page.

LLM Prompt



Python + Flask Implementation

```
# app.py
from flask import Flask, render_template, request, redirect, url_for
import uuid
import openai # Requires `pip install openai`

app = Flask(__name__)
openai.api_key = 'your-openai-api-key' # Replace with your actual API key

# In-memory store (for demo; use a real DB for production)
posts = {}

@app.route('/submit', methods=['GET'])
def submit_form():
    return render_template('submit.html')

@app.route('/analyze', methods=['POST'])
def analyze_post():
    title = request.form['title']
    content = request.form['content']
    tags = request.form['tags'].split(',')

    prompt = (
        "You are an academic quality assistant. Analyze the following student-submitted content "
        "for clarity, depth, relevance, and usefulness as a study resource.\n\n"
        f"\"{content}\" \n\n"
        "Rate the content from 1-10 and explain your reasoning."
    )

    try:
        response = openai.ChatCompletion.create(
            model='gpt-3.5-turbo',
            messages=[{"role": "user", "content": prompt}],
            temperature=0.7,
            max_tokens=300
        )
        analysis = response["choices"][0]["message"]["content"]
    except Exception as e:
        analysis = f"Error from LLM: {str(e)}"

    post_id = str(uuid.uuid4())
    posts[post_id] = {
        'title': title,
```

```

        'content': content,
        'tags': tags,
        'analysis': analysis
    }

    return redirect(url_for('show_result', post_id=post_id))

@app.route('/result/<post_id>', methods=['GET'])
def show_result(post_id):
    post = posts.get(post_id)
    if not post:
        return "Post not found", 404
    return render_template('result.html', post=post)

if __name__ == '__main__':
    app.run(debug=True)

```

DEMO - April 15

tags and results from the rubric

make a page that displays the tags and the resulting ranking from the rubric

flow:

submit post>>loading page>>page that displays the tags and rubric

parse the syllabus and get the topics, then tag the post with what topic it covers, and display to the poster the quality of their.

architecture diagram deliverable

prompt testing document - testing prompt performance

to do:

demo video

api request (takes in text in the box and grades it and make suggestions) a colored rating is displayed on the post after it is rated

show code and demo

demo majority of the given time to present

make sure to talk about user stories - sally wants a study partner

introduce francis and her interests

remind to hold questions until the end

Demo walkthrough sections (we can use videos or just screenshots):

- signing in and creating a post. (upvotes and downvotes for sentiment analysis) (Annarose)

- Tagging resources (Theresa)

- Comparing your notes to the lecture slides (Abhiram)

- Quality analysis (Erik)

- Feedback - the page where it displays the LLM rubric results (Elizabeth)

Pages that I created for the demo (code in github etc):

StudyBuzz

LoginRegister

Login

Email

Password

Sign In

Register

StudyBuzz

LoginRegister

Create an Account

Username

Email Address

Password

Confirm Password

Register

Already have an account? [Login here](#)

Create an Account

Username

Email Address

Password

Confirm Password

Register

Already have an account? [Login here](#)

Create an Account

Username

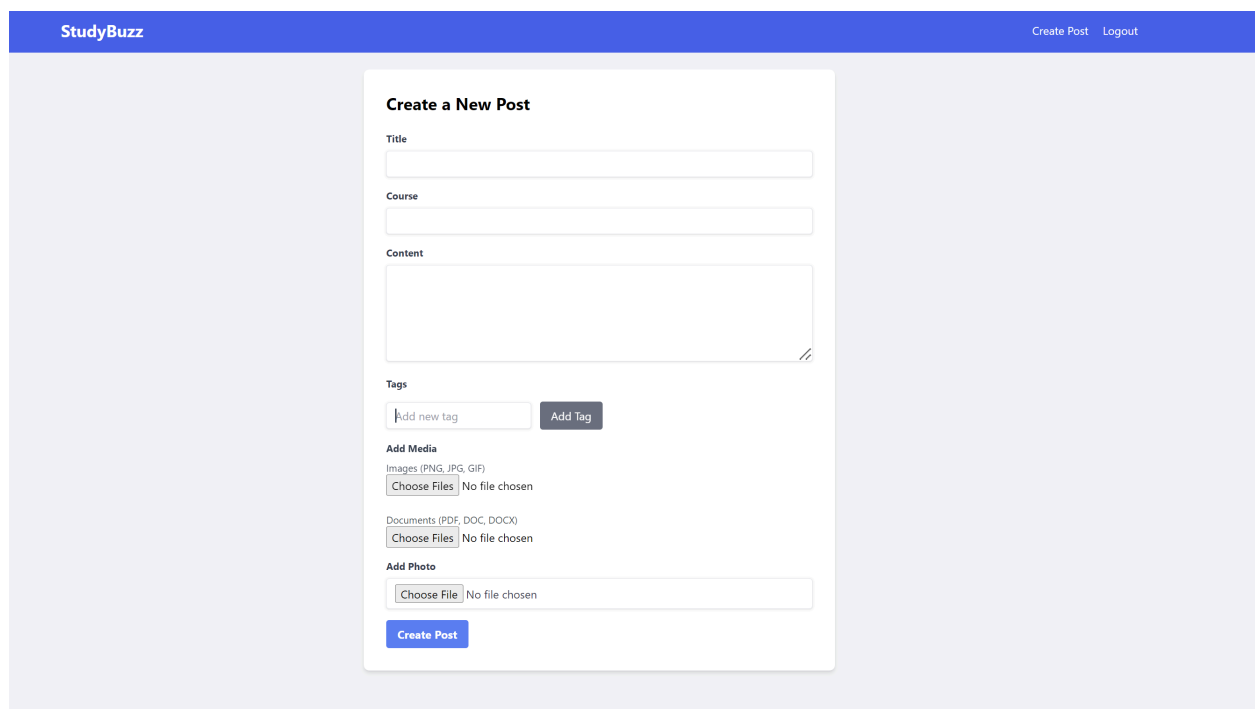
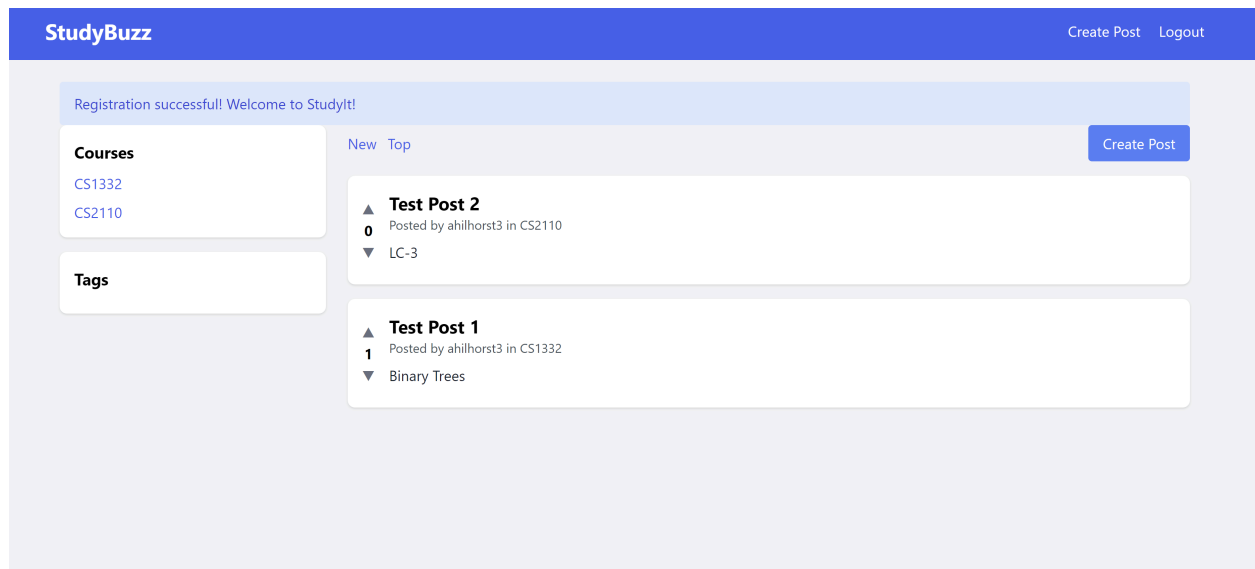
Email Address

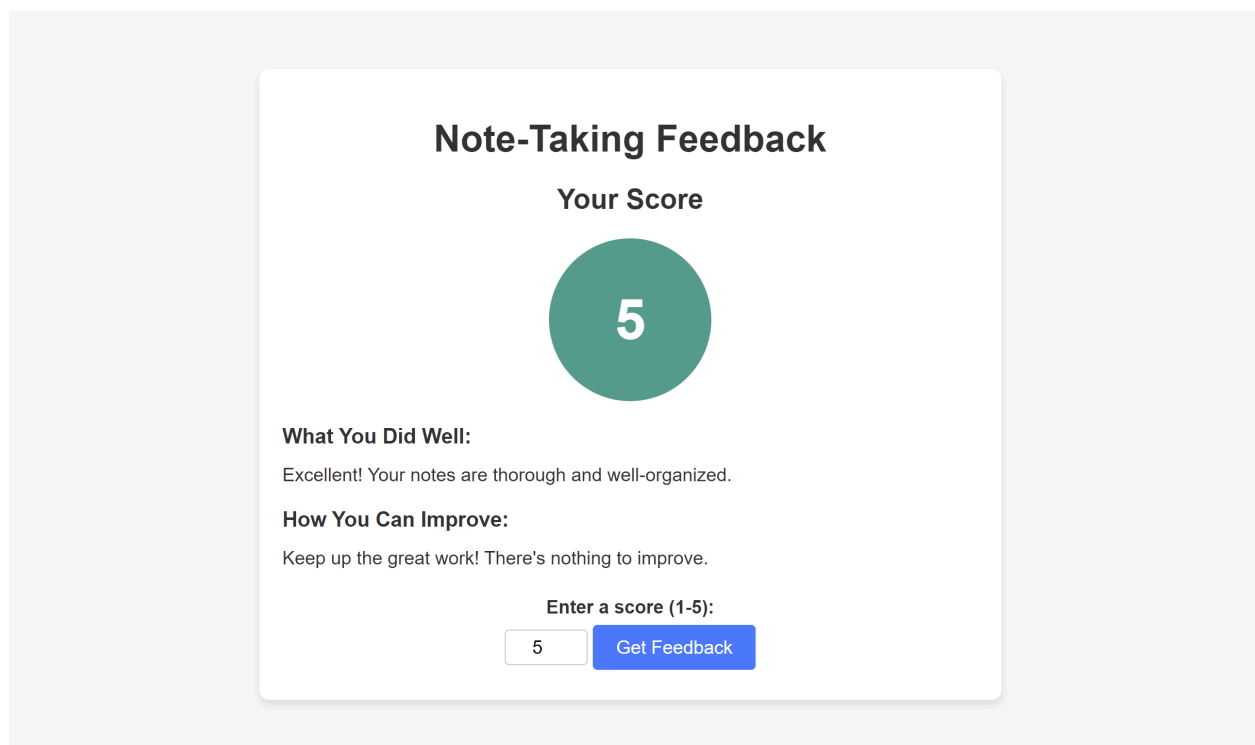
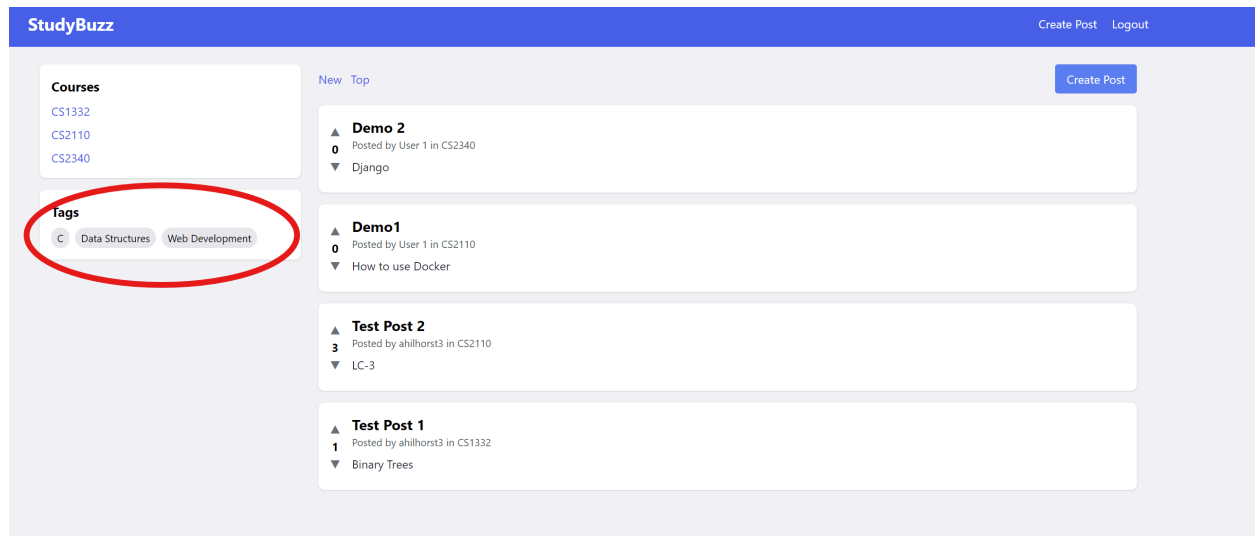
Password

Password must be at least 8 characters long and contain at least one letter and one number

Confirm Password

Register





Note-Taking Feedback

Your Score

3

What You Did Well:

You have a decent structure in your notes!

How You Can Improve:

You could add more detailed explanations and examples.

Enter a score (1-5):

Get Feedback

StudyBuzz LLM Topic Tagging!

Paste your text document below, and the system will tag it with relevant topics from the course syllabus.

Paste your document here...

Tag Document

Detected Topics:

No topics detected yet.

StudyBuzz LLM Topic Tagging!

Paste your text document below, and the system will tag it with relevant topics from the course syllabus.

1332 notes from today's lecture:

1. Arrays

Fixed size, contiguous memory

Example: [1, 2, 3, 4, 5]

Access time: $O(1)$

Tag Document

Detected Topics:

Arrays

Stacks

Queues

Trees

Graphs

Recursion

<https://youtu.be/uP2--Jjji0w>

DEMO MC

what is a VIP

what is this course

what is PNAMBC

defining the problem

what:

shy

language barrier

other commitments/and overloaded schedule

who:

students professors TAs

where:

in class and study environments

why:

to improve grades

to improve community

improve mastery and contribution

how:

by crowdsourcing resources and facilitating